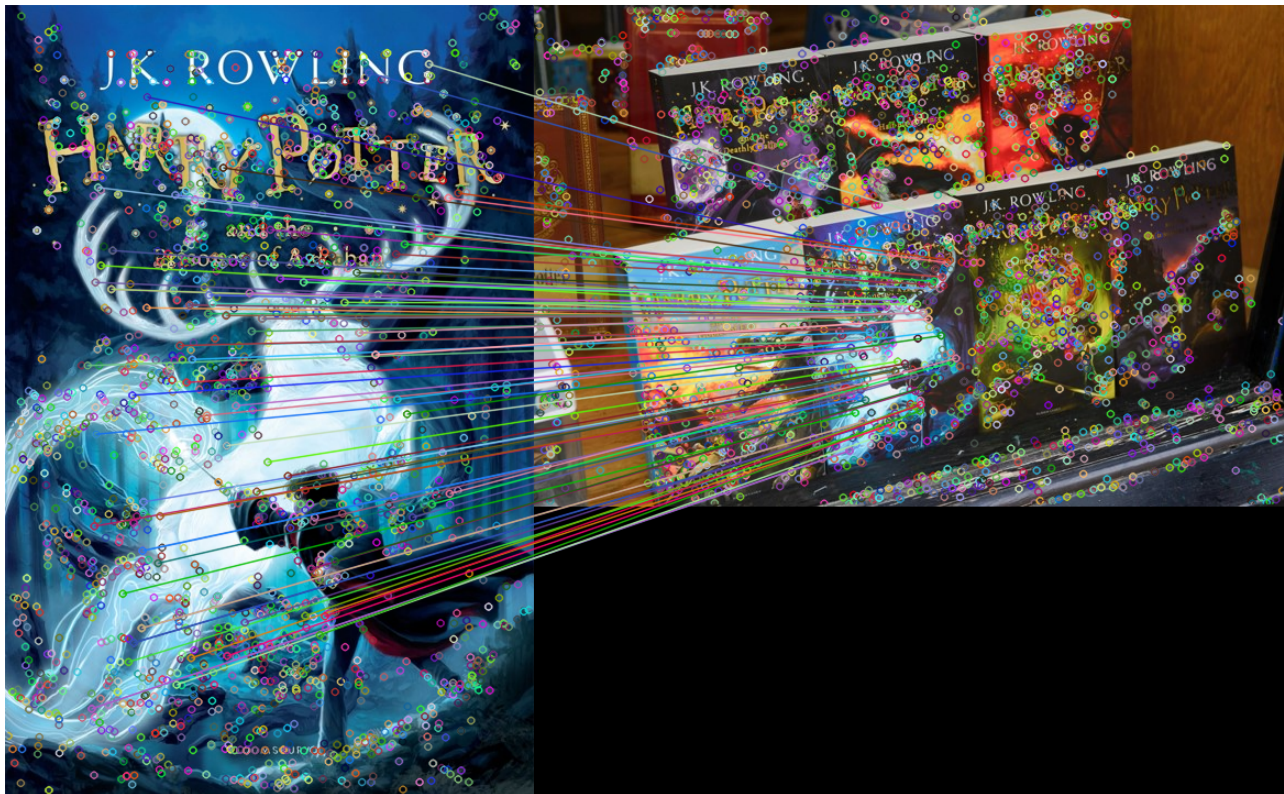


#016 Feature Matching methods comparison in OpenCV

👤 [Strahinja Zivkovic](#) 📁 [OpenCV](#) 📅 13.01.2021 | 💬 0



Highlights: Hi. Welcome to our new lecture. In our previous posts, we already explained in great detail how to extract distinct [features](#) (also called keypoints) from an image. Now we will learn how to compare two or more images by extracting pairs of identical feature points from those images. To accomplish this, we can apply several different feature matching methods that OpenCV provides. We hope that this post will complete your knowledge in this area and that you will become an expert for feature matching in OpenCV. So, let's begin with our post.

Tutorial Overview:

1. [Introduction](#)
2. [Brute Force Matching with ORBdetector](#)
3. [Brute-Force Matching with SIFT detector and Ratio test](#)
4. [FLANN based Matcher](#)

1. Introduction

First, let's remind ourselves how we can extract and match features from two images.

Humans have a natural ability to recognize different objects. We can do this because our brain is triggered by the most distinct features of an image. In image processing, these feature points will assist us to compare and detect objects in images or videos.

So, let's say that we have the following two images. Our goal is to extract keypoints from the image on the left side, which we usually call a target image. For that, we will use techniques like a corner, edge detection, or contour detection. Then, by using feature matching algorithms we can find all the matches in an image on the right.



In the example above on the left side, we can see the picture of Daniel-Day Lewis. This is our target image. On the right side, we can see this target image within the photo in which Daniel poses with his colleges after they won an Oscar. Notice that the image on the left side is a rotated and scaled version of Daniel's image on the right side. You may wonder how can we detect matches if one image is not the exact copy of the other image? Well, we can detect keypoints in both images, and then the feature matching algorithm will find their matches. This means that we can match features even if the target image is rotated and has a different size

To extract the features from an image we can use several common feature detection algorithms. In this post we are going to use two popular methods: Scale Invariant Feature Transform (SIFT), and Oriented FAST and Rotated BRIEF (ORB). For feature matching, we will use the Brute Force matcher and FLANN-based matcher. So, let's begin with our code.

2. Brute-Force Matching with ORB

detector

In this chapter, we are going to extract features using **Oriented FAST and Rotated BRIEF (ORB)** detector and we will use the Brute-force method for feature matching. First, let's import the necessary libraries and load our images. Also, we will convert images into grayscale.

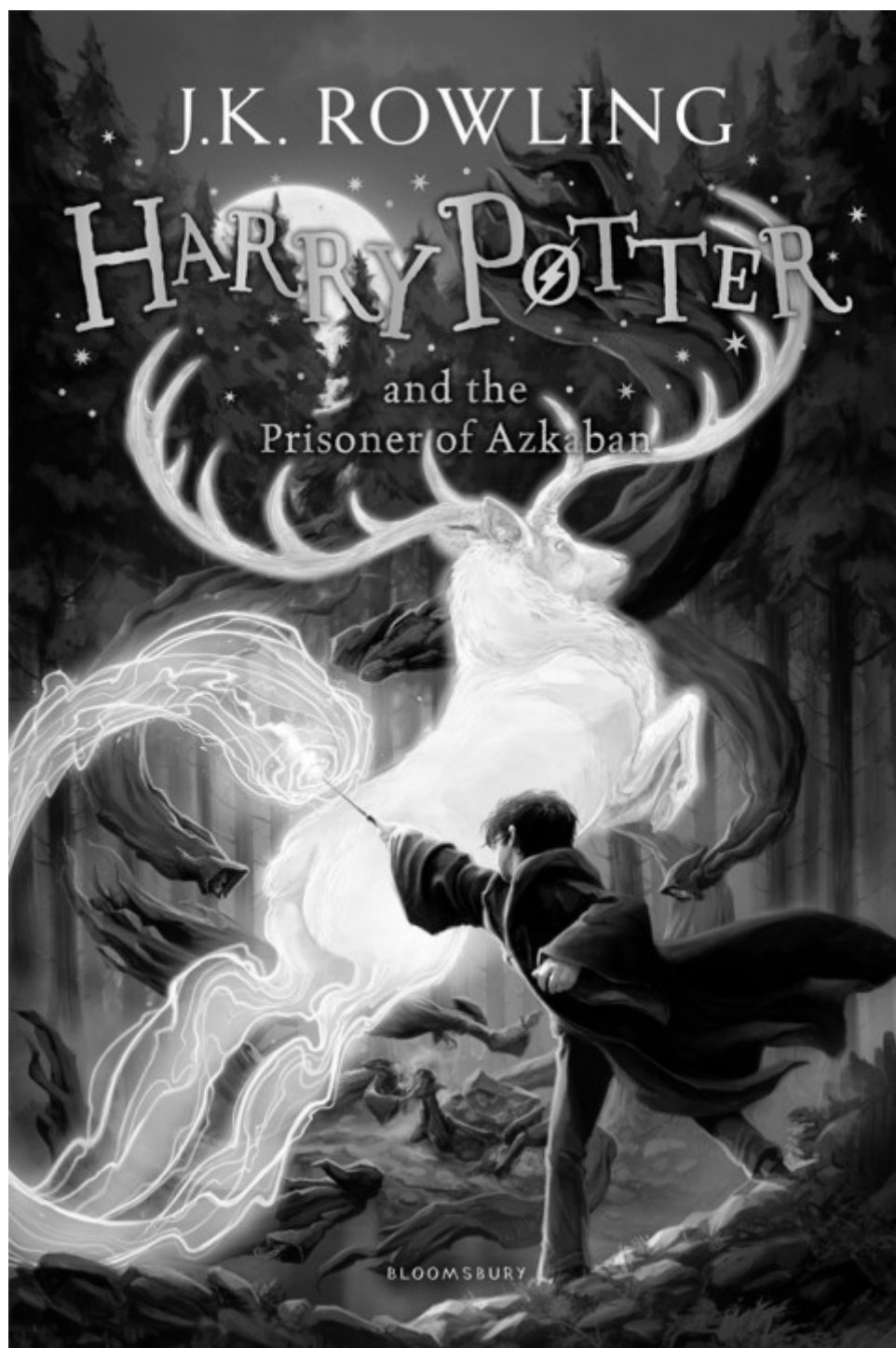
```
import cv2
import numpy as np
from matplotlib import pyplot as plt

from google.colab.patches import cv2_imshow

img1 = cv2.imread("Picture1.jpg")
img2 = cv2.imread("Picture2.jpg")

img1_gray = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
img2_gray = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

cv2_imshow(img1_gray)
```

So, as you can see the target image is a cover of a *Harry Potter and Prisoner of Azkaban*. Now, let's show the second image.

```
cv2_imshow(img2_gray)
```




Here we can see a picture of a bookshelf with several different books. As you can see, the image is taken from a non-frontal angle. That is a potential problem. The other problem that may occur is that all books have the same headline. This also may affect our feature matching. Nevertheless, let's continue with our code.

Now, we're going to apply the Brute-Force matching with ORB descriptors. First, we need to create the ORB detector using the function `cv2.ORB_create()`.

```
# Create our ORB detector and detect keypoints and descriptors
orb = cv2.ORB_create()
```

This is now our detector object. Next, we will detect keypoints and descriptors using the function `orb.detectAndCompute()`. We are going to pass two parameters. The first parameter is the input image and the second parameter is the `mask`. In our case, we are going to pass `None` because we don't need a mask here. Then, we are going to do the same thing for the second image.

```
# Find keypoints and descriptors with ORB
keypoints1, descriptors1 = orb.detectAndCompute(img1, None)
keypoints2, descriptors2 = orb.detectAndCompute(img2, None)
```

The next step is to create the `BFMatcher` object using the `cv2.BFMatcher_create()` function. This function consists of an optional parameter `normType` that specifies the distance as a measurement of similarity between two descriptors. For binary string based descriptors like

ORB, we usually use `cv.NORM_HAMMING`. This parameter calculates the Hamming distance between the arrays. **The Hamming distance between two strings of equal length is the number of positions at which the corresponding symbols are different.** To better understand this distance metric have a look at the following image.

The Hamming distance = 1

17001154 17101154

The Hamming distance = 2

29003681 29053081

The Hamming distance = 3

58812749 58672340

The second parameter is `crossCheck`. By default, it is set to `False`. In this case, `BFMatcher` will find the k nearest neighbors for each query descriptor. On the other hand, if `crossCheck==True`, then the `knnMatch()` method will return only the best matches. It will return matches with values (i, j) such that i^{th} descriptor in a set A (descriptors from the first image) has j -th descriptor in a set B (descriptors from the second image) as the best match and vice-versa.

```
# Create a BFMatcher object.
# It will find all of the matching keypoints on two images
bf = cv2.BFMatcher_create(cv2.NORM_HAMMING, crossCheck=True)
```

Now, we can check where the matches occur by using the function `bf.match()`. Then, we are going to pass descriptors of the first image, and descriptors of the second image.

```
matches = bf.match(descriptors1, descriptors2)
```

So, now we have our matches. The next step is to sort them according to their distance. In our code, we will use the function `sorted()`. It has one required parameter `iterable` (in our case that are our matches) and several optional parameters. Another parameter that we use in our code is `key`. We use this parameter to decide the sort order (by default, this argument is

set to None). In our case, we will use `lambda x:x.distance` expression. This expression will return the sorted list containing the items from the matches. In that way, the items will be sorted in ascending order of their distances so that best matches (with low distance) come to the front.

To better understand this let's have a look at these matches. All matches have a lot of different attributes like a training index query index, but more importantly, they have a distance attribute. So, we can print the distance of a one-match using the following code.

```
single_match = matches[0]
single_match.distance
```

Output:

62.0

So, the smaller distance means that it is a better match, and the longer distance means that it is a less likely match. Therefore, with the following expression, we're going to sort our matches by that distance attribute.

```
matches = sorted(matches, key=lambda x:x.distance)
```

Now, let's continue with our post. The next step is to draw our matches using the function `draw_matches()`. This function consists of the following parameters:

- **img1, img2** – the input images
- **keypoints1** – keypoints detected in the first image
- **keypoints2** – keypoints detected in the second image
- **matches** – all detected matches

In case you don't want to draw all matches you can choose the best ones at the top of the list of matches. We will choose the first 30 by indexing parameter `matches` in our function, The next parameter we're going to provide is `mask`. We are going to pass `None` because we don't want to use a mask here. And then finally, we'll set parameter `flags=2`. In that way, we will define how to draw the matching points.

```
ORB_matches = cv2.drawMatches(img1, keypoints1, img2, keypoints2, matches[:30], None, flags=2)
cv2.imshow(ORB_matches)
```

Output:



As you can see, our matches do not look so great. We can conclude that ORB descriptors don't always work. There are a couple of reasons for that. One issue is that the book in the second image is taken at an angle, and also, we have several other similar books in the same picture. Therefore, we're going to choose some more sophisticated method of feature matching.

3. Brute-Force Matching with SIFT detector and Ratio test

Now, we are going to run a similar code. However, this time we're going to do is use **Scale Invariant Feature Transform (SIFT)** descriptors. This method is perfectly suitable for our goal because **it detects features that are invariant to image scale and rotation**. Moreover, features are local and based on the appearance of the object in certain interesting points. They are also robust to changes in illumination, noise, and minor changes in viewpoint.

SIFT was first presented in 2004, by David G. Lowe from the University of British Columbia in the paper, Distinctive Image Features from Scale-Invariant Keypoints, This algorithm was patented several years ago, and since then it is included in the non-free module in OpenCV. That is why we need to install the older version of OpenCV because SIFT is not included in the new OpenCV library.

Note, that the patent for SIFT expired last year, so the algorithm potentially can be used for commercial purposes.

```
!pip install opencv-python==3.4.2.16
!pip install opencv-contrib-python==3.4.2.16
```

Now we will continue by creating a SIFT object with the function

```
cv2.xfeatures2d.SIFT_create()
```

```
# Create our SIFT detector and detect keypoints and descriptors
sift = cv2.xfeatures2d.SIFT_create()
```

Then, just like we did with ORB, we're going to find the keypoints and descriptors with SIFT.

```
# Find the key points and descriptors with SIFT
keypoints1, descriptors1 = sift.detectAndCompute(img1, None)
keypoints2, descriptors2 = sift.detectAndCompute(img2, None)
```

Next, we're going to calculate these matches using the Brute-Force method.

```
bf = cv2.BFMatcher()
```

Now, we're going to calculate our matches using a slightly different function `bf.knnMatch()`.

This function finds the k best matches of number for each descriptor from a query set. Let's see what this actually means.

To visualize this let's print our descriptors.

```
descriptors1
```

Output:

```
array([[ 19.,  17.,  32., ...,  0.,  0.,  0.],
       [  0.,  0.,  0., ..., 64.,  3.,  5.],
       [ 15.,  0.,  0., ..., 22.,  0.,  2.],
       ...,
       [ 89.,  33.,  0., ...,  0., 14., 16.],
       [ 17.,  9.,  0., ...,  0.,  0.,  0.],
       [  1., 17., 117., ...,  0.,  0.,  0.]], dtype=float32)
```

```
descriptors2
```

Output:

```
array([[ 58.,  2.,  2., ...,  0.,  0.,  0.],
       [  0.,  0.,  0., ...,  0.,  1.,  2.],
       [  3.,  2.,  3., ...,  0.,  0.,  2.],
       ...,
       [ 72.,  6.,  2., ...,  2.,  0.,  0.],
       [  0.,  0.,  0., ..., 116.,  4.,  6.],
       [  0.,  0.,  0., ..., 15.,  0.,  0.]], dtype=float32)
```

Here we can see descriptors in the first and in the second image. The function `bf.knnMatch()` provides two matches for each of these descriptors when the parameter `k=2`.

```
matches = bf.knnMatch (descriptors1, descriptors2,k=2)
```

```
matches
```

Output:

```
[[<DMatch 0x7fcdf4cd9350>, <DMatch 0x7fcdf4cd94b0>],
 [ <DMatch 0x7fcdf4cd9390>, <DMatch 0x7fcdf4cd90d0>],
 [ <DMatch 0x7fcdf4cd94f0>, <DMatch 0x7fcdf4cd92f0>],
 [ <DMatch 0x7fcdf4cd93f0>, <DMatch 0x7fcdf4cd9430>],
 [ <DMatch 0x7fcdf4cd9490>, <DMatch 0x7fcdf4cd9410>],
 [ <DMatch 0x7fcdf4cd9450>, <DMatch 0x7fcdf4cd93d0>],
 [ <DMatch 0x7fcdf4cd9470>, <DMatch 0x7fcdf4cd95d0>],
 ...]
```

Here we can see pairs of these matches. The first match in the first column is a better match than the match in the second column. Note that if `k=3` we would have the third-best match and so on.

To keep only the strong matches we will use David Lowe's ratio test. Lowe proposed this ratio test in order to increase the robustness of the SIFT algorithm. The goal of this test is to get rid of the points that are not distinct enough. The general idea is that there needs to be enough difference between the first best match and the second-best matches.

Now by using indexing we can extract one first and one second-best match and compare their distance measurements.

```
AA1 = matches[131][0]
AA1.distance
```

```
163.8322296142578
```

```
AA2 = matches[131][1]
AA2.distance
```

```
291.624755859375
```

Here we can see that distance of the first best match is far away from the distance of the second match. Therefore, the entire descriptor of that point is probably distinct enough and it should be a good match.

On the other hand, if the first best match is pretty close to the second match, then this point probably is not distinct enough.

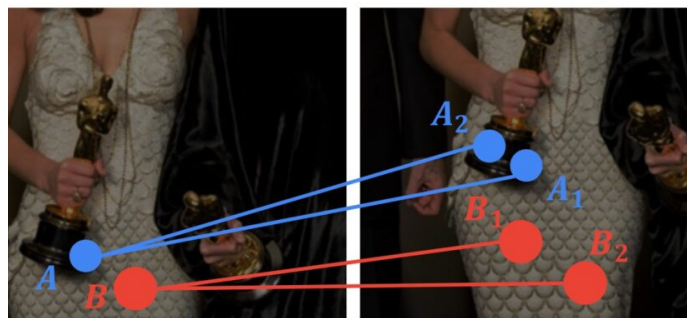
```
BB1 = matches[1][0]
BB1.distance
```

```
287.2438049316406
```

```
BB2 = matches[1][1]
BB2.distance
```

```
301.8923645019531
```

To better understand this have a look at the following image.



$AA_1 = 20$ - The best match

$AA_2 = 80$ - The second best match

$BB_1 = 10$ - The best match

$BB_2 = 25$ - The second best match

True

False

if $20 < 0.6 \cdot 80$
 $20 < 48$

if $10 < 0.6 \cdot 25$
 $20 < 15$

Here we can see two feature points A and B in the first image. We will match descriptors of these points with the first best match and the second best match in a second image. Lowe's test then checks that the two distance measurements are sufficiently different. If they are, that point is preserved. On the other hand, if they are not distinct enough, then the keypoint is eliminated and will not be used for further calculations.

Now, we can apply David Lowe's ratio test.

```
good_matches = []

for m1, m2 in matches:
    if m1.distance < 0.6*m2.distance:
        good_matches.append([m1])
```

So, we created a loop that defines which matches will be discarded and which matches will be preserved. So, if $m1$ distance is less than 60% of $m2$ distance, then the descriptor for that

particular row will be preserved. On the other hand, if `m1` distance is greater than 60% of `m2` distance, then it's probably not a good match, and the descriptor for that particular row will be discarded. Therefore less distance means a better match.

Now, let's print our `good_matches` .

```
good_matches
```

Output:

```
[(<DMatch 0x7fcdf4cd9a30>],
 [<DMatch 0x7fcdf4cd9b90>],
 [<DMatch 0x7fcdf4cc3230>],
 [<DMatch 0x7fcdf4cc3290>],
 [<DMatch 0x7fcdf4cc3350>],
 [<DMatch 0x7fcdf4cc52f0>],
 [<DMatch 0x7fcdf4cc5b10>],
 ...
```

Also, we can print the length of that list. We can see that we actually discarded a large number of poor matches. In the initial list, we had 2342 matches and we ended up with only 116 best matches.

```
len(matches)
```

Output:

2342

```
len(good_matches)
```

Output:

116

Now it's time to draw these matches and see how they performed. This time we will use a similar function `cv2.drawMatchesKnn()` . We will pass the same parameters as we did with the ORB detector.

```
SIFT_matches =cv2.drawMatchesKnn(img1, keypoints1, img2, keypoints2, good_matches, None,
flags=2)
cv2_imshow(SIFT_matches)
```

Output:



As you can see with the SIFT detector we obtained much better results as compared to the ORB detector. However, this is still not a perfect result. The problem is that headlines on all books are identical. So, if we have many similar images, it could still be an error in the actual matching.

4. FLANN based Matcher

Using a FLANN based matcher instead of a Brute-Force matcher we're going to introduce a more complex parameter drawing mechanism, that allows us to draw only the clear matches.

First, we are going to detect features with SIFT and this part of the code will be identical to the previous one.

```
sift = cv2.xfeatures2d.SIFT_create()
```

```
keypoints1, descriptors1 = sift.detectAndCompute(img1, None)
keypoints2, descriptors2 = sift.detectAndCompute(img2, None)
```

Now, we need to define the FLANN parameters. FLANN stands for Fast Library for Approximate Nearest Neighbors. It contains a collection of algorithms optimized for fast nearest neighbor search in large datasets and for high dimensional features. It works faster than BFMatcher for large datasets. However, it's not going to find the best possible matches. Instead, it's just going to find good matching candidates.

In order to try to increase the precision or the quality of the matches, we can play with FLANN parameters. However, that will slow down the algorithm. That is why we will use the following

default parameters for a FLANN matcher.

To construct FLANN parameters we will use a **k-dimensional tree** which is the alternative way to organize data structures. In computer science, a k-dimensional tree is a space-partitioning data structure for organizing points in a k-dimensional space.

We will set `FLAN_INDEX_KDTREE = 0` . Also, we are going to define two sets of parameters: index parameters, and search parameters. In the `index_params` we will create a dictionary by passing `FLAN_INDEX_KDTREE` into the `dict()` dictionary algorithm. We will also set the number of trees which in our case is equal to 5. Finally, we will say that `search_params` is equal to the dictionary, and we'll set parameter `checks=50` .

```
FLAN_INDEX_KDTREE = 0
index_params = dict (algorithm = FLAN_INDEX_KDTREE, trees=5)
search_params = dict (checks=50)
```

So, now we have our parameters. Next, we create the FLANN based matcher object using the function `cv2.FlannBasedMatcher()` . As parameters, we will pass index parameters as well as the search parameters.

```
flann = cv2.FlannBasedMatcher(index_params, search_params)
```

To calculate matches we will use `flann.knnMatch` the *k* nearest neighbor matches. Next, we will create a similar ratio test as we did with the SIFT detector.

```
matches = flann.knnMatch (descriptors1, descriptors2,k=2)
```

```
good_matches = []
```

```
for m1, m2 in matches:
    if m1.distance < 0.5 * m2.distance:
        good_matches.append([m1])
```

Now, when we have our good matches it's time to draw them. Note, that parameter `flags=2` and we can also set this parameter to `flags=0` . In the first case, that function will draw only lines between matching points.

```
flann_matches =cv2.drawMatchesKnn(img1, keypoints1, img2, keypoints2, good_matches, None,
flags=2)
cv2_imshow(flann_matches)
```

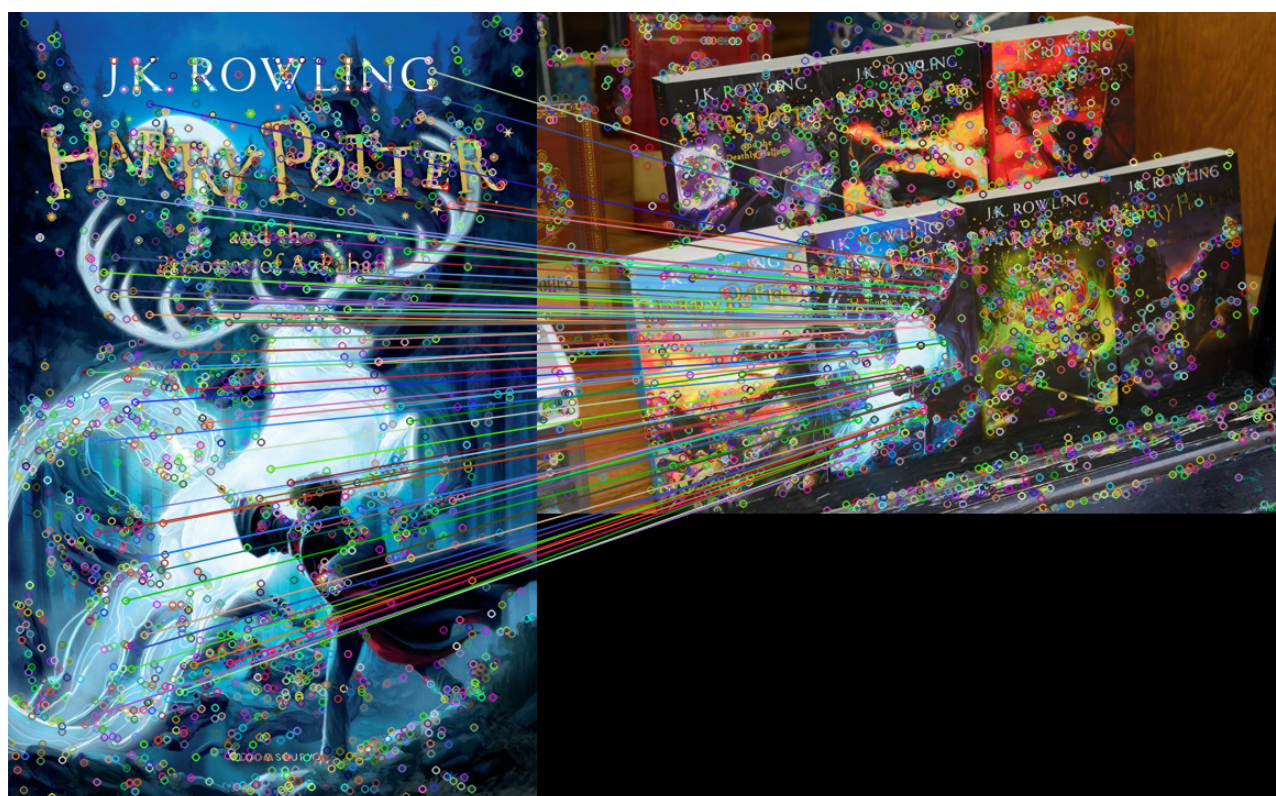
Output:



On the other hand, in the second case, all points will be shown as you can see in the example below.

```
flann_matches = cv2.drawMatchesKnn(img1, keypoints1, img2, keypoints2, good_matches, None,
flags=0)
cv2.imshow('flann_matches')
```

Output:



Now let's pass several additional parameters that will help us to better visualize our matches. We can draw these lines in one specific color, and then show all those points from `flag=0` as a different color. The way we can do that is by simply adding the parameter `mask`. So, the first several lines of code will be identical.

```
sift = cv2.xfeatures2d.SIFT_create()

keypoints1, descriptors1 = sift.detectAndCompute(img1, None)
keypoints2, descriptors2 = sift.detectAndCompute(img2, None)

FLAN_INDEX_KDTREE = 0
index_params = dict (algorithm = FLAN_INDEX_KDTREE, trees=5)
search_params = dict (checks=50)

flann = cv2.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch (descriptors1, descriptors2, k=2)
```

After that, we are going to do create a new object called `matchesMask`.

```
matchesMask = [[0,0] for i in range(len(matches))]
```

If we take a look at this object we can see that it is a matrix that consist of only zeros.

```
matchesMask
```

Output:

```
[[0, 0],
 [0, 0],
 [0, 0],
 [0, 0],
 [0, 0],
 [0, 0],
 [0, 0],
 [0, 0],
 ...]
```

Now, we want to change some of these zeros to one, depending on if we have a good match or not. To do that we need to slightly modify our ratio test. We will change `m1` and `m2` to a single tuple by putting parentheses around it. And then we will enumerate our matches. In that way, we can actually keep track of the index marker. Then, we can create the mask by setting `matchesMask` at index `i` to be equal to `[1,0]`. By doing this we are going to label lines where we actually have a good match as 1. So, all we're doing is we have this giant list of zeros.

```
for i,(m1, m2) in enumerate (matches):
    if m1.distance < 0.5 * m2.distance:
        matchesMask[i] = [1,0]
```

Now we're going to use this mask to create a drawing parameter dictionary. Then, we specify the

color of single points and the color of the line. Next, we're going to do is say pass in the `matchesMask` which defines where we had a match and where we just have a single point. Finally, we will set `flags=0` because we want to show all these single points.

```
draw_params = dict (matchColor = (0,0,255), singlePointColor = (0,255,0), matchesMask =  
matchesMask, flags=0 )
```

Now, we need to make just a few corrections to the function `cv2.drawMatchesKnn()` First, instead of `good_matches` we will pass in all matches. In that way instead of appending matches to a list to a list `good_matches`, we will use `matchMask` and `draw_params` to filter them out based on their color. Also, instead of parameter flags, we will write `**draw_params`.

```
flann_matches =cv2.drawMatchesKnn(img1, keypoints1, img2, keypoints2, matches,  
None, **draw_params)  
cv2.imshow('flann_matches')
```

Output:



As you can see we obtained pretty accurate result using this method.

Here you can download code from our [GitHub repo](#).

Summary

In this post, we learned how to match feature points using three different methods: Brute Force matching with ORB detector, Brute-Force Matching with SIFT detector, and FLANN based matcher. We demonstrate which of these feature matching methods provide the most accurate results. In the next post, we will discuss object detection with MeanShift and CamShift algorithms.